

↳ maximum sum
of non-adjacent elements.

↳ let's try out all subsequences
with the given
constraint

↙
pick the one
with the
maximum
sum.

↳ problem

↳ pick / not-pick

↳ pre-requisites

↳ print all subsequences.

↳ contiguous / non-contiguous
sequence which follows
the order.

ans $\rightarrow$ (3, 1, 2)

{ }

3

1

2

3

1

1

2

3

2

3

1

2

but the

order

is

preserved.

↳ Recursion

(3, 1, 2)

(3, 2)

✓ ✗ ✓

3

2

3

1

✗

✓

✓

→ index
take / not take

n.t, t, t

arr $\rightarrow$ [3, 1, 2]

0, 1, 2

f(ind, c)

{

if (ind > n)

print(c)

return;

c.add(arr[i])

f(ind+1, c); $\rightarrow$ take

c.remove(arr[i])

f(ind+1, c) $\rightarrow$ not take

}

does
not
take

the
sequence

↳ Back to the main question.

↳ Pick subsequence element with
no adjacent.

f(ind)

f

if (ind == 0)

{

return a[ind];

}

if (ind < 0) return 0;

pick = a[ind] + f(ind-2);

not pick = 0 + f(ind-1);

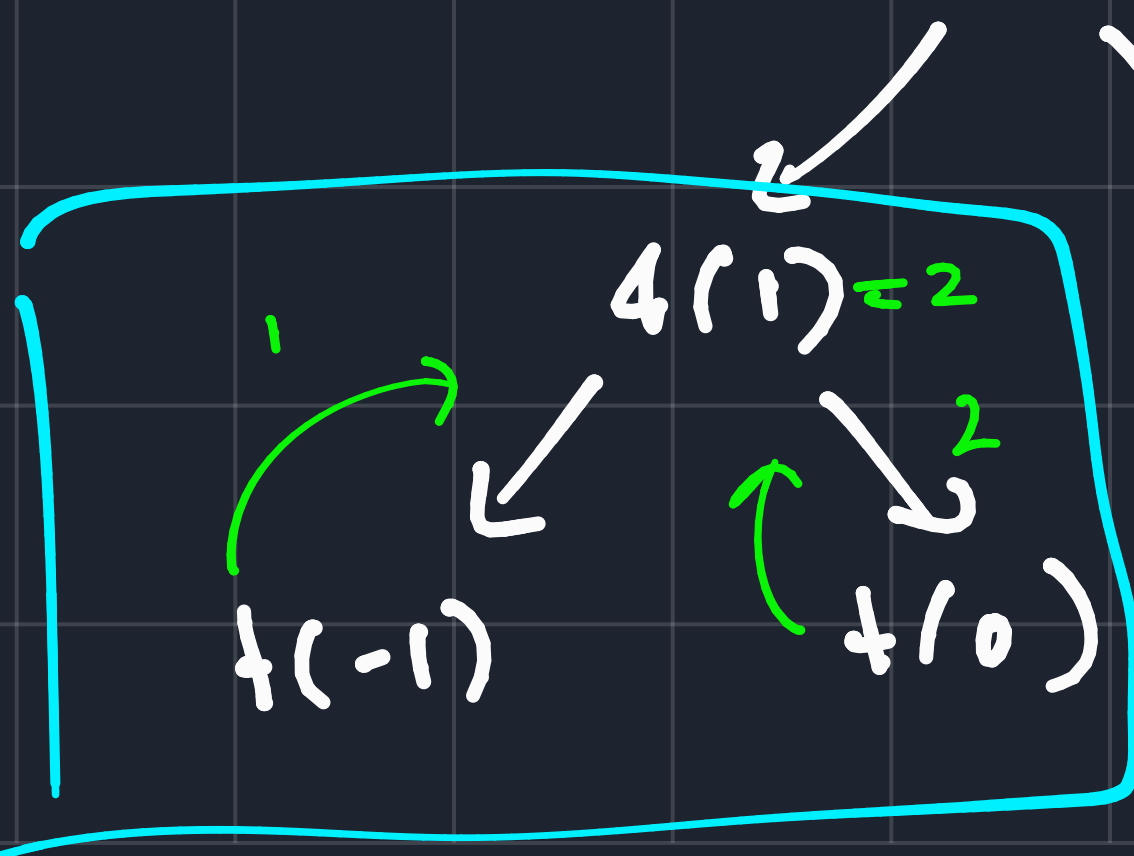
return max(pick, not pick)

}

(2 1 4 2)

f(3) → sum [0...3]

f(3) → 11



f(2) = 2



we are reaching

here because we

did not pick

index 1.

memoize

it.

tabulation

↳

dp[n]

uses tabulation & space optimization
see the code.